GemaMat

An Intelligent Assistant for Atomic Layer Deposition Workflows

Final Undergraduate Project Presentation

Dabeet Kumar Das (240314)
MSE 496 | Department of Materials Science and Engineering
Indian Institute of Technology Kanpur
Supervised by Prof. Shikhar Misra

Dataset synthesis **Structured extraction** **Agentic RAG**

Motivation

Why Atomic Layer Deposition?

- Atomic Layer Deposition (ALD) is a precise vapor phase technique used to deposit ultra thin films on complex 3D structures.
- ALD finds applications in various domains such as the semiconductor industry, where it enables deposition of ultra-thin films for gate oxides, DRAM capacitors and other advanced memory devices. It is also widely used in photovoltaics, nanoelectronics, optics, and biomedical systems, making ALD a highly versatile and important technique for modern systems.

Core problem

Literature is rich but unstructured

Despite its vast application in the semiconductor industry, there exists no centralized knowledge repository for the methods and processes in atomic layer deposition, making the knowledge of the domain highly fragmented across literature.

Project target

An end-to-end assistant

GemaMat integrates domain specific knowledge from studies on atomic layer deposition and leverages large language models for providing context aware guidance for queries related to ALD.

Project Objectives

OBJECTIVE 1

Build a reliable ALD corpus

The first objective towards a centralized solution for ALD workflows was to acquire the fuel of the system - data. This objective focused on collecting process records, retrieve PDFs, preserve metadata, and convert the literature into simpler data formats (like txt files) for downstream tasks.

OBJECTIVE 2

Extract structured scientific fields

The second objective was the construction of knowledge repository, where a researcher can seamlessly access various methods and processes to deposit the material of their choice. This objective focused on capturing target materials, process chemistry, deposition conditions, film properties, and characterization outputs from the extracted data and then converting them into structured JSON schemas.

OBJECTIVE 3

Enable grounded scientific QA

The third objective was to build an advanced and intelligent agentic system capable of QnA on the entire corpus. This objective focused on chunking the extracted data, upserting the chunks to a vector database and setting up a RAG enabled pipeline for QnA.

Objective 1: Building the ALD Corpus

Primary source

- This project uses the AtomicLimits ALD database as the core source of processes and references.
- AtomicLimits loads the data using exposed API endpoints to raw JSON files containing DOIs for all the papers. They however, do not have any PDFs stored at server side.
- **Problem:** The metadata for material names and reactants are mismatched on AtomicLimits.

SciHub

Source for raw PDFs

Collected DOIs were stored and SciHub was used to retrieve the corresponding PDFs using basic HTML scraping.

Metadata CSVs

Operational traceability

Proper metadata was saved during extraction from SciHub to try a multi API approach later on, which however didn't seem to work out.

Dataset Preparation Pipeline

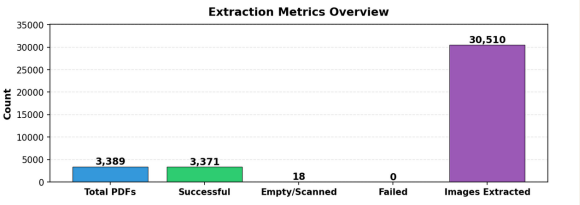
What happens after PDF retrieval

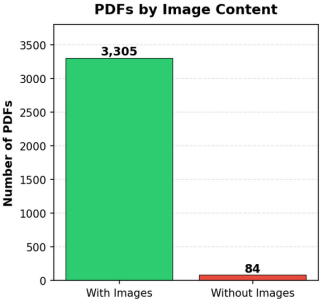
- Each paper is converted into a dedicated folder with `content.txt` and an `Images/` directory.
- The preprocessing code uses `pypdf`, regex cleaning, and PIL-based image extraction to preserve visual context.
- Small decorative images are filtered out with explicit size thresholds before storage.

Folderized assets

Good downstream ergonomics

The project stores every paper in a structure that can be reused by extraction scripts, upload utilities, and the dashboard. This corpus was uploaded to Kaggle to ensure reproducibility in this work.





The GitHub repository includes extraction visualizations that summarize how PDF processing progressed over the corpus.

Quality Control and Final Corpus

3,389

Working paper corpus

Large-scale extraction milestone reached during corpus building.

3,335

Clean papers retained

Final usable set after dataset sanctity checks and fixes.

98.4%

Retention rate

Only 54 papers were discarded after quality review.

Why?

- A preprocessing bug initially stripped Miller indices because bracketed scientific expressions were mistaken for citation markers.
- A note on dataset quality: This project focuses on the development of a knowledge "corpus" and requires clean, high quality data to ensure that the agentic frameworks in other sections of the pipeline are able to work properly. The final interactive chat interface also requires high quality to be actually useful to a user.

Objective 2: Extracting Structured Scientific Fields

Extraction Philosophy

- The extraction layer converts each paper into JSON objects that can drive search, visualization, comparison, and filtering.
- Prompts explicitly require null handling and evidence grounding to reduce hallucination pressure.

DESIGN CHOICE

Paper-level decomposition

A single paper becomes a set of structured sections such as summary, target material, deposition conditions, film properties, and characterization.

```
{
  "target_material": "Al2O3",
  "process_type": "thermal ALD",
  "main_precursors": ["TMA", "H2O"],
  "temperature_range": "150-250",
  "summary": "...",
  "evidence": "..."
}
```

JSON-first

Reusable outputs

The extraction schema is designed for reuse by databases, dashboards, and retrieval systems.

Eight Specialist Extraction Agents

Agent roster

- Summary
- Target material
- Precursor / co-reactant / purge
- Deposition conditions
- Reaction mechanism
- Substrate information
- Film properties
- Characterization

MODEL STRATEGY

Local Llama-family extraction

This project uses LlaMA 3.2 3B for extracting all the JSON schemas. The inference is done on a Ubuntu 24.04 system using Ollama for local inference.

Parallel paper processing

Faster structured extraction

Agents are executed in parallel per paper, then their JSON outputs are written into a paper-specific output folder.

Citation-based prompts

Scientific Grounding

Prompts request exact supporting text when possible so outputs remain inspectable and grounded.

Modular outputs

Easy to browse and seed

The resulting JSON files are directly useful for MongoDB seeding, dashboard rendering, and retrieval metadata.

Validation and Robustness Features

How the project reduces extraction drift

- Schema-aware defaults prevent malformed outputs from collapsing downstream pipelines.
- Validation code checks top-level types, missing keys, wrong field types, and empty objects.
- Evidence strings are tested against the original text to detect unsupported claims.

CHECK 1

Schema conformity

Every extraction family has a defined default object and expected field types.

CHECK 2

Evidence containment

The validator checks whether the claimed evidence actually appears in the source full text.

CHECK 3

Graceful failure

Due to a smaller model being used for extraction, malformed responses were only flagged and the pipeline was not broken.

Objective 3: Enabling Grounded QnA

Index construction

- Paper text is chunked into 300-word passages with 50-word overlap before embedding.
- Each vector stores text plus metadata such as `paper_id`, `target_material`, and `process_type`.
- The project uses a Pinecone index named `ald-llamamat`.

INDEXING INSIGHT

Metadata matters

Good metadata lets retrieval stay scoped to papers, materials, and process types instead of becoming a generic semantic search demo.

300

Chunk size

Word-based chunks are large enough to preserve scientific context.

50

Chunk overlap

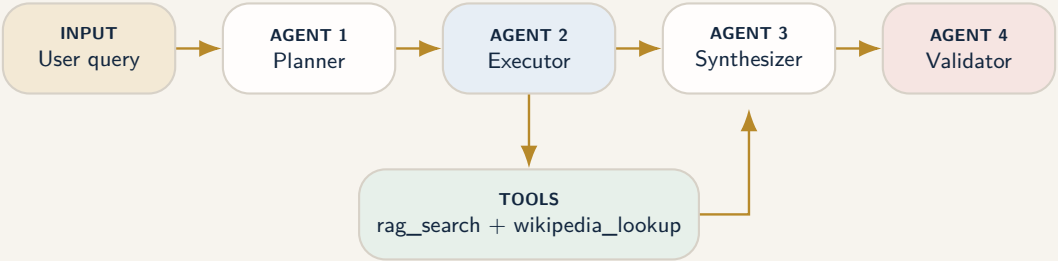
Overlap reduces boundary loss when important facts span adjacent passages.

Pinecone Inference

Embedding pipeline

The repo uses `llama-text-embed-v2` for embeddings and prepares vectors for batched upsert.

Agentic RAG Architecture



WHY THIS IS IMPORTANT

Reasoning is separated from execution

The ReWOO-style pattern keeps planning explicit, tool calls deterministic, and token usage more efficient.

TRACEABILITY

The API exposes the agent trace

The backend returns answer, sources, diagnostics, plan, execution, and validation artifacts while preserving frontend compatibility.

The Executor (No LLM)

Planner JSON → Directed Acyclic Graph (DAG) → Central Result Store

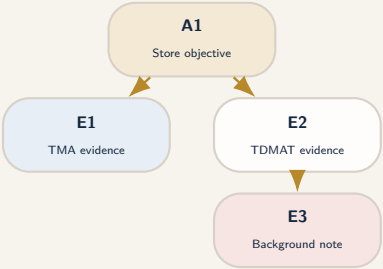
PLANNER JSON

A1	analysis
E1	rag_search (TMA)
E2	rag_search (TDMAT)
E3	wikipedia_lookup

A1 unlocks E1 and E2.

E2 unlocks E3.

Directed Acyclic Graph (DAG)



RESULT STORE

```
after A1 {A1}
after E1 {A1, E1}
after E2 {A1, E1, E2}
after E3 {A1, E1, E2, E3}
```

Each completed step appends one more artifact.

KEY IDEA

The executor is deterministic

Retrieval Mechanics

STEP 1

HyDE query transformation

Generate a hypothetical domain-matched passage to bridge the gap between user language and scientific literature.

STEP 2

Reciprocal-rank fusion

Combine the original-query list and the HyDE list so agreement across both retrieval views is rewarded.

STEP 3

Reranking and final evidence

Use a reranker to prioritize the most relevant chunks before synthesis and validation.

What this achieves

- Better semantic coverage than plain vector search alone.
- More reliable scientific evidence selection before answer generation.
- A retrieval stack designed around grounded reasoning rather than generic chatbot behavior.

User Interface

TIER 1

Material library

Browse the literature by target material rather than scanning paper titles one by one.

TIER 2

Paper archive

View the set of papers attached to a chosen material system.

TIER 3

Paper dossier

Inspect extracted properties, evidence, and the original source PDF for a selected paper.

Interface design choices

- MongoDB Atlas stores the structured paper records used by the dashboard.
- Vercel Blob stores PDFs so users can inspect the original literature alongside extracted fields.
- The interface combines browsing, dossier-style inspection, and chat-based question answering in one workflow.

Chat Panel

The left-hand chat panel provides an interactive interface for querying the corpus. Users can ask natural-language questions and receive context-grounded answers streamed back in real time, with source attribution linking each response to the originating paper.

Implementation Stack

Pipeline Layers

Layer	Components
Acquisition	Python scraping scripts, DOI resolution, metadata logs
Preparation	`pypdf`, regex cleaning, PIL image filtering, audit scripts
Extraction	Quantized Llama-family local models, Ollama validation utilities
Retrieval	Pinecone vector index, HyDE, reranking, FastAPI backend
Interface	Next.js App Router, React, TailwindCSS, Recharts
Data services	MongoDB Atlas and Vercel Blob

Model Stack

Primary LLM — Google Gemini
`gemini-3.0-flash-preview`

Embedding — Pinecone Inference
`llama-text-embed-v2`

Reranking — Pinecone Inference
`bge-reranker-v2-m3`

Extraction — Ollama
`LlaMA-3.2-3B-Instruct`

Limitations

What can be improved

- PDF access constraints still limit how complete the acquisition stage can become. Specifically, access to APIs like Elsevier, Wiley and other journals is required to build an extensive dataset.
- Figure understanding remains partial because multimodal coverage was compute-bound. During experimentation, Qwen-3.5-8B-VL was used to summarise graphs, scientific diagrams, but due to storage and extraction-scale constraints it was not done on the entire corpus.
- The repo does not yet include a full human-annotated benchmark for formal extraction scoring.
- Production use depends on external services and API keys for MongoDB, Vercel Blob, Pinecone, and Gemini. Deploying to full production scale would require migration to a more stable cloud service like AWS/GCP. Further, as the vector database size expands, costs would be incurred for Pinecone as well.

Thank You

Open to questions and feedback

Dabeet Kumar Das

dabeetkd24@iitk.ac.in ▪ https://github.com/DabeetDas/ald-llamamat